Cloud IAM Primer

Identity and Access Management in an AWS-style Cloud

CIS*6510 — Assignment: Cloud IAM Audit

This primer (generated using Claude) teaches the cloud and IAM concepts you need for the assignment. It assumes you can program in Python and know the course's Modules 1–3 (threat modelling, cryptography, authentication), but *no* prior cloud experience. Everything is offline: you will never touch a real AWS account, spend money, or need credentials. The assignment implements a *simplified* model of AWS IAM; wherever this primer describes real AWS, the assignment PDF states precisely which parts you must implement and which are out of scope.

Contents

1 What "the cloud" is, in two minutes **1**

1.1 The shared-responsibility model 2

2 IAM: the objects **2**

2.1 ARNs: naming everything 3

3 Policy anatomy **3**

4 How a request is decided **4**

5 Conditions **4**

6 Roles, trust policies and temporary credentials **5**

6.1 STS and `sts:AssumeRole` 5

7 The instance metadata service and SSRF (the Capital One lesson) **5**

8 Least privilege and `iam:PassRole` **6**

9 The privilege-escalation catalogue **7**

10 Tools you will meet later (pointers only) **7**

11 Check your understanding **8**

1 What "the cloud" is, in two minutes

A *cloud provider* such as Amazon Web Services (AWS) rents you computing resources — virtual machines, storage, databases, functions — over the Internet, billed by use. The US National Institute

of Standards and Technology defines cloud computing around on-demand self-service, resource pooling and rapid elasticity.[1]

The resources you will meet in this assignment:

- **S3** (Simple Storage Service): object storage. Data lives in *buckets*; each object has a *key* (its path). Think "a giant, globally-named key/value store for files."
- **EC2** (Elastic Compute Cloud): virtual machines ("instances").
- **Lambda**: run a function without managing a server.
- **KMS**: managed encryption keys.

1.1 The shared-responsibility model

AWS secures the infrastructure — the hardware, the hypervisor, the data-centre. *You* secure what you put on it: your data, who can access it, and how your software is configured. AWS calls this security "*of*" the cloud versus security "*in*" the cloud.[2] Access control — the subject of this assignment — is squarely *your* responsibility, and it is where a very large share of real cloud breaches begin.

2 IAM: the objects

Identity and Access Management (IAM) is the service that answers one question for every request: *is this principal allowed to perform this action on this resource?* Its building blocks:

- **User**: a long-lived identity for a person or program. Has credentials (a password and/or access keys).
- **Group**: a named bundle of users. Policies attached to a group apply to every user in it. A group is *not* something that can make requests — it only exists to attach permissions to many users at once.
- **Role**: an identity with permissions but *no* long-term credentials. Instead, a trusted principal *assumes* the role and receives temporary credentials. Roles are how one identity (or an AWS service such as EC2) temporarily acts with a different set of permissions.
- **Policy**: a JSON document listing permissions. A *managed* policy is a standalone, named, reusable document that can be attached to many identities; an *inline* policy is embedded directly in one identity and belongs only to it.[3]

An identity's **effective permissions** are the union of *every* policy that applies to it. For a user that means: policies attached directly, that user's inline policies, and — for each group the user belongs to — the group's attached and inline policies. For a role: its attached and inline policies. (The assignment's data model section pins this down exactly, including the JSON field names.)

[1] P. Mell and T. Grance, *The NIST Definition of Cloud Computing*, NIST SP 800-145, 2011. https://csrc.nist.gov/pubs/sp/800/145/final

[2] *AWS Shared Responsibility Model*. https://aws.amazon.com/compliance/shared-responsibility-model/

[3] *Managed policies and inline policies*. https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_managed-vs-inline.html

2.1 ARNs: naming everything

Every AWS resource has an **Amazon Resource Name (ARN)**, a colon-delimited string. You will see, among others:

- `arn:aws:iam::123456789012:user/alice` — the user `alice`
- `arn:aws:iam::123456789012:role/DeployRole`
- `arn:aws:iam::123456789012:group/Admins`
- `arn:aws:s3:::my-bucket` and `arn:aws:s3:::my-bucket/path/key`
- `arn:aws:ec2:us-east-1:123456789012:instance/*`

The 12-digit number is the *account ID*. In ARNs, resource names are case-sensitive.[4]

3 Policy anatomy

A policy document is a list of **statements**. Each statement grants or denies a set of actions on a set of resources, optionally only when certain conditions hold. Annotated example:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",                        # Allow or Deny
      "Action": ["s3:GetObject", "s3:List*"],  # what you may do
      "Resource": "arn:aws:s3:::reports/*",     # on which resources
      "Condition": {                            # optional tests
        "IpAddress": {"aws:SourceIp": "203.0.113.0/24"}
      }
    }
  ]
}
```

- **Effect** is `Allow` or `Deny`.
- **Action** names service operations as `service:Operation`. Action matching is *case-insensitive*: `s3:GetObject` and `S3:getobject` are the same.[5] The wildcard `*` matches any run of characters and `?` matches one; `s3:Get*` covers `s3:GetObject`, `s3:GetBucketPolicy`, and so on.
- **Resource** names ARNs. Matching is *case-sensitive*, with the same `*` and `?` wildcards. `arn:aws:s3:::reports/*` matches every object in the `reports` bucket.
- **Condition** (optional) adds tests on the request; see below.

[4] *IAM JSON policy elements: Resource.* https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_elements_resource.html

[5] *IAM JSON policy elements: Action.* https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_elements_action.html

4 How a request is decided

When a principal makes a request, AWS gathers every applicable statement and applies three rules, in order. By default everything is denied; a request must be explicitly allowed; and an explicit `Deny` beats any `Allow`.[6] In the simplified model you will implement:

Decision procedure. Collect all of the principal's effective statements.

1. If *any* matching statement has effect `Deny`, then the result is **DENY**, with the reason reported as `explicit_deny`.
2. Otherwise, if *any* matching statement has `Effect: Allow`, the result is **ALLOW** with reason `explicit_allow`.
3. Otherwise the result is **DENY** with reason `implicit_deny`.

A statement "matches" when its Action matches the request action, its Resource matches the request resource, *and* its Condition (if any) is satisfied.

This "default deny, explicit deny wins" design is deliberate: you can hand out a broad `Allow` and then carve out exceptions with a narrow `Deny` that can never be accidentally overridden.

5 Conditions

A `Condition` block makes a statement apply only in certain request contexts. Each entry is `operator: { key: value }`. The assignment uses exactly four operators and three keys:

- Operators: `Bool`, `StringEquals`, `StringLike` (wildcards), and `IpAddress` (CIDR match).
- Keys:
  - `aws:MultiFactorAuthPresent` — was MFA used on this request?
  - `aws:SourceIp` — the caller's IP address
  - `aws:RequestedRegion` — the target AWS region

Two rules you must get right:

- `Bool` values are the *strings* `"true"`/`"false"`, exactly as in real AWS.
- If a condition names a key that is *absent* from the request context, the comparison is **false**, so the statement does not apply.[7] This holds for `Allow` *and* `Deny` statements: a `Deny` that is conditioned on a missing key simply does not fire. (Real AWS has an `...IfExists` variant that behaves differently; it is out of scope.)

When one key lists several values they are OR-ed; several keys or operators are AND-ed.

[6] *Policy evaluation logic* and *How AWS enforcement code ... evaluates requests.* https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic_policy-eval-denyallow.html

[7] *IAM JSON policy elements: Condition operators.* https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_elements_condition_operators.html

Why "missing key = false" matters for security. Suppose you try to *require* MFA by writing a `Deny` with `Bool aws:MultiFactorAuthPresent = false`. A request made with a stolen long-term access key carries *no* MFA key at all — so your `Deny` never fires and the attacker is *not* blocked. To actually require MFA in this model you must instead *Allow* only when `aws:MultiFactorAuthPresent = true`; with no MFA key present, no Allow matches and the request is implicitly denied. You will use exactly this pattern in Part A3.

6 Roles, trust policies and temporary credentials

A role carries *two* policies that do very different jobs:

- its **permission policies** (attached + inline) say *what the role can do* once assumed;
- its **trust policy** says *who is allowed to assume it*. The trust policy is the one place in this assignment where a policy names a `Principal`.

A `Principal` in a trust policy can be an AWS identity (`{"AWS":"arn:aws:iam::123456789012:user/alice"}`), the account "root" (`{"AWS": "arn:aws:iam::123456789012:root"}`, meaning *any* identity in that account), a wildcard (`"*"` or `{"AWS": "*"}`, meaning *anyone* — almost always a mistake[8]), or an AWS *service* (`{"Service": "ec2.amazonaws.com"}`, letting that service assume the role).

6.1 STS and `sts:AssumeRole`

The Security Token Service (STS) hands out *temporary* credentials. Calling `sts:AssumeRole` on a role returns a short-lived access key, secret and session token that carry *the role's* permissions.[9] Whether a call succeeds is a **two-sided** decision:

1. the *caller's* own identity policies must allow `sts:AssumeRole` on the role's ARN (with no explicit deny), *and*
2. the *role's trust policy* must allow the caller as a principal.

Miss either side and the call fails. (Real AWS is slightly more lenient within a single account: if the trust policy names your user ARN directly, it can grant the assumption even without side (1). The assignment deliberately *always* requires both sides.) A service principal (`ec2.amazonaws.com`) in the trust policy lets that *service* assume the role — which is the hinge of several escalation techniques below.

7 The instance metadata service and SSRF (the Capital One lesson)

An EC2 instance that needs AWS credentials gets them from a role attached to the instance. Software on the instance reads them from the **Instance Metadata Service (IMDS)** at the link-local

[8] *IAM JSON policy elements: Principal.* https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_elements_principal.html

[9] *AWS STS `AssumeRole` API.* https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html

address `169.254.169.254`; the role's temporary credentials appear under `/latest/meta-data/iam/security-credentials/<role-name>` and are rotated automatically.[10]

This convenience is also a liability. In a **Server-Side Request Forgery (SSRF)** attack, an attacker tricks a server into making a request *the attacker chooses*. If a vulnerable web app on an EC2 instance can be coerced into fetching `http://169.254.169.254/...`, it will return the instance role's credentials to the attacker — who can then use them from anywhere.

This is, in outline, the 2019 **Capital One** breach. Public court filings describe a misconfigured firewall that let commands reach a server, which obtained credentials for a role (redacted in the indictment as `*****-WAF-Role`) and used them to list and copy data from storage buckets; roughly 100 million people were affected.[11] Independent technical analyses attribute the chain to SSRF against the metadata service combined with an over-privileged instance role.[12]

Two defences matter here, and both appear in the assignment's writeup question:

- **Least privilege on the role.** Had the instance role been scoped to only the buckets it needed, the stolen credentials would have reached far less.
- **IMDSv2.** A hardened metadata service requires a session token obtained by an HTTP `PUT` and sets the response hop limit to 1, which defeats most SSRF attempts that can only issue simple `GET` requests.[13]

8 Least privilege and `iam:PassRole`

Least privilege means giving every identity only the permissions it genuinely needs. AWS's own guidance leads with starting from minimal permissions and granting more only as required.[14] Most of the escalation paths you will hunt exist because some identity has *more* than it needs.

One permission deserves special attention: `iam:PassRole`. To launch an EC2 instance or create a Lambda function *with* a role attached, you must be allowed to "pass" that role to the service. If you can pass a powerful role to a service you control, you can make that service act with the role's permissions — so `iam:PassRole` is dangerous unless it is scoped to a specific, safe set of roles.[15] Two controls contain it: restrict *which* roles can be passed (the `Resource` on the `iam:PassRole` statement), and restrict *to which service* with the `iam:PassedToService` condition key.

---

[10] *Retrieve security credentials from instance metadata.* https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-metadata-security-credentials.html

[11] U.S. DOJ, W.D. Wash., press releases, 17 Jun 2022 and 4 Oct 2022. https://www.justice.gov/usao-wdwa/pr/former-seattle-tech-worker-convicted-wire-fraud-and-computer-intrusions; and the criminal complaint, Case 2:19-mj-00344 (filed 29 Jul 2019).

[12] N. Neto, S. Madnick et al., *A Case Study of the Capital One Data Breach*, MIT, 2020. https://cams.mit.edu/wp-content/uploads/capitalonedatapaper.pdf; and B. Krebs, *What We Can Learn from the Capital One Hack*, 2019. https://krebsonsecurity.com/2019/08/what-we-can-learn-from-the-capital-one-hack/

[13] C. MacCárthaigh, *Add defense in depth against . . . SSRF . . . EC2 Instance Metadata Service*, AWS Security Blog, 19 Nov 2019. https://aws.amazon.com/blogs/security/defense-in-depth-open-firewalls-reverse-proxies-ssrf-vulnerabilities-ec2-instance-metadata-service/

[14] *Security best practices in IAM.* https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

[15] *Grant a user permissions to pass a role to an AWS service.* https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_use_passrole.html

9 The privilege-escalation catalogue

A *privilege escalation* is a sequence of allowed actions that leaves the attacker with *more* power than they started with — ideally, administrator. Below is the finite catalogue you must detect in Part A2. Each item lists the permissions it needs and a three-line sketch of *why it works*. The precise resource ARNs for each required request are specified in the assignment. The catalogue is drawn from well-known public research.[16]

CreateAccessKey on another user. Needs `iam:CreateAccessKey` on the victim user. *Why:* a new access key *is* working credentials for that user; you now act as them, with all their permissions.

Create/UpdateLoginProfile on another user. Needs `iam:CreateLoginProfile` (or the `Update` variant) on the victim. *Why:* you set or reset their console password and log in as them.

AssumeRole (two-sided). Needs `sts:AssumeRole` on a role whose trust policy also allows you. *Why:* you receive the role's temporary credentials; if the role is powerful, so are you.

PassRole + RunInstances. Needs `ec2:RunInstances` plus `iam:PassRole` on a role that EC2 can assume. *Why:* you launch an instance with that role attached and read its credentials from the metadata service.

PassRole + CreateFunction + InvokeFunction. Needs `iam:PassRole` on a role Lambda can assume, plus `lambda:CreateFunction` and `lambda:InvokeFunction`. *Why:* your function runs *as* the role; invoking it runs your code with the role's permissions.

UpdateAssumeRolePolicy + AssumeRole. Needs `iam:UpdateAssumeRolePolicy` on a role plus the ability to assume it. *Why:* you rewrite the role's trust policy to trust *you*, then assume it.

Self-grant (user). Needs any one of these on yourself (or on a policy you are attached to): `iam:AttachUserPolicy`, `iam:PutUserPolicy`, `iam:AddUserToGroup` into an admin group, or `iam:CreatePolicyVersion` with `iam:SetDefaultPolicyVersion`. *Why:* each lets you attach, inline, inherit or redefine a policy that grants yourself administrator.

Self-grant (role). Needs `iam:AttachRolePolicy` or `iam:PutRolePolicy` on a role, used against itself. *Why:* the role grants *itself* administrator permissions.

An escalation is often a *chain*: if Alice can create an access key for Bob, and Bob can assume an admin role, then Alice → Bob → admin is a two-hop path even though Alice has no admin permission herself. Part A2 asks you to find the *shortest* such path for every principal that has one.

10 Tools you will meet later (pointers only)

Real audits use automated tooling. You are *not* using these in the assignment — you are building a tiny version of what they do — but they are worth knowing:

- **Cloudsplaining** — scans IAM policies for least-privilege violations and produces a risk report (BSD-3). https://github.com/salesforce/cloudsplaining
- **PMapper** — models principals as a graph and searches for privilege-escalation paths, much like your Part A2 (AGPL-3). https://github.com/nccgroup/PMapper

[16] S. Gietzen, *AWS IAM Privilege Escalation — Methods and Mitigation*, Rhino Security Labs. https://rhinosecuritylabs.com/aws/aws-privilege-escalation-methods-mitigation/

- **Parliament** — lints IAM policy documents for errors and risky patterns (BSD-3). https://github.com/duo-labs/parliament

11 Check your understanding

Try these before you start; answers overleaf.

1. A user matches one `Allow` statement and one `Deny` statement for the same request. What is the decision?
2. A statement's `Condition` requires `aws:SourceIp` in a CIDR range, but the request context has no `aws:SourceIp` at all. Does the statement apply?
3. Alice has `sts:AssumeRole` on `AdminRole` in her own policy, but `AdminRole`'s trust policy lists only {`"Service": "ec2.amazonaws.com"`}. Can Alice assume it?
4. Why is `iam:PassRole` on a powerful role, combined with `ec2:RunInstances`, enough to escalate?
5. Name one single-line change to the Capital One instance role that would most have limited the blast radius.
6. You want to require MFA for a destructive action. Explain why a `Deny` conditioned on `aws:MultiFactorAuthPresent` being `false` is the wrong way to do it in this model.

Answers.

1. **DENY** (`explicit_deny`): an explicit `Deny` always beats an `Allow`.
2. No. A key named in a condition but absent from the context makes the comparison false, so the statement does not apply.
3. No. `AssumeRole` is two-sided; the trust policy trusts only the EC2 service, not Alice, so side (2) fails.
4. You launch an instance with that role attached and read its temporary credentials from the metadata service, then act with the role's permissions.
5. Scope the role's permissions (and/or its `Resource`s) to only the specific buckets it needed — i.e. replace broad S3 access with least privilege. (Enforcing IMDSv2 is also accepted.)
6. A stolen long-term key carries no MFA context key, so the condition is false and the `Deny` never fires. Instead, *Allow* the action only when `aws:MultiFactorAuthPresent = true`.